16. String Matching

Finding patterns hidden inside text.

Previous Section:

15. Hash Tables

Contents:

1. What is String Matching?
2. String Matching Algorithms
    2.1. Brute-Force Algorithm
    2.2. Rabin-Karp Algorithm
    2.3. Knuth-Morris-Pratt (KMP) Algorithm
Summary
References

Every day, we search for information without even thinking about it.

When you press **Ctrl + F** to find a word in a document, search for a message in your chat history, look up a gene sequence in biology, or type a query into a search engine, the computer is performing the same fundamental task—it is trying to find one string of characters inside another.

At first glance, this may seem like a simple problem. Why not just compare every character until a match is found?

For small pieces of text, that works perfectly. But modern applications rarely deal with small amounts of data. Search engines index billions of webpages, DNA databases contain billions of genetic characters, and cybersecurity systems continuously inspect massive streams of network traffic. Searching every possible position one by one quickly becomes too slow.

This is where **String Matching** comes in. String Matching is one of the most fundamental topics in computer science. It provides efficient techniques for locating patterns inside larger texts, allowing computers to search quickly even when processing enormous amounts of information.

In this section, we will learn what String Matching is, why it is important, and explore several algorithms that solve this problem with different strategies and levels of efficiency.

1. What is String Matching?

String Matching is the process of finding whether a **pattern** (also called a substring) appears inside a larger **text**. Instead of comparing the entire document, the goal is to determine whether a specific sequence of characters exists, where it occurs, and sometimes how many times it appears.

For example, suppose we have:

- **Pattern (P):** `"abc"`
- **Text (T):** `"abdabcadbc"`

The objective is to determine whether the pattern `"abc"` appears in the text. In this example, the pattern is successfully found beginning at the fourth character of the text.

Although this example is small, real-world applications often involve texts containing millions or even billions of characters. Searching efficiently therefore becomes a significant computational challenge.

String Matching forms the foundation of many modern software systems because searching for text is one of the most common operations performed by computers.

Some important real-world applications include:

- **Plagiarism Detection** – Comparing documents against existing sources to identify copied or highly similar content.
- **DNA Sequencing** – Finding specific genetic patterns within extremely long DNA sequences to support biological research and medical diagnosis.
- **Digital Forensics** – Searching large collections of digital evidence for keywords, file signatures, or suspicious patterns during investigations.
- **Spell and Grammar Checking** – Detecting incorrect spellings, grammar mistakes, and inappropriate word usage by comparing text against known language patterns.
- **Spam Filtering** – Identifying emails containing suspicious keywords or phrases to determine whether a message is likely to be spam.
- **Search Engines and Database Searching** – Locating documents, webpages, or records that contain user-specified keywords, enabling fast information retrieval.
- **Intrusion Detection Systems (IDS)** – Examining network packets for known malicious signatures or attack patterns to detect potential cybersecurity threats.

Because searching is such a fundamental operation, many specialized string matching algorithms have been developed. Some focus on simplicity, while others are designed to minimize unnecessary comparisons, making them suitable for processing massive datasets efficiently.

Throughout this section, we will explore these algorithms and understand how each improves the efficiency of searching in different situations.

2. String Matching Algorithms

Many algorithms have been developed to solve the String Matching problem. Although they all share the same objective—finding a pattern inside a larger text—they differ significantly in how they perform the search.

Some algorithms compare characters directly, some transform strings into numerical values, while others preprocess the pattern to avoid unnecessary comparisons.

In this section, we will study three of the most fundamental string matching algorithms:

- **Brute-Force Algorithm** – the simplest and most straightforward approach.
- **Rabin-Karp Algorithm** – improves searching using rolling hash values.
- **Knuth-Morris-Pratt (KMP) Algorithm** – avoids repeated comparisons by utilizing information within the pattern itself.

2.1. Brute-Force Algorithm

The **Brute-Force Algorithm** (also known as the **Naïve String Matching Algorithm**) is the simplest approach to solving the string matching problem.

The idea is straightforward:

> Start from the beginning of the text and compare the pattern character by character. Whenever a mismatch occurs, move the pattern one position to the right and start comparing again from the beginning of the pattern.

Since every possible starting position is checked, the algorithm is guaranteed to find the pattern if it exists.

The overall time complexity is $O(n \times m)$ where $n =$ length of the text; $m =$ length of the pattern

Suppose we are given:

- Text (X) = "abcdabcabcdf"
- Pattern (Y) = "abcdf"

We use two pointers:

- **i** → points to the current character in the **text**
- **j** → points to the current character in the **pattern**

```
Text (X)
Index : 0 1 2 3 4 5 6 7 8 9 10 11
        a b c d a b c a b c  d  f
        ↑
        i

Pattern (Y)
Index : 0 1 2 3 4
        a b c d f
        ↑
        j
```

- Compare the characters.
  - `X[0] = 'a'` and `Y[0] = 'a'` → Match ✓. Increment both pointers (`i = 1; j = 1`).
  - `X[1] = 'b'` and `Y[1] = 'b'` → Match ✓. `i = 2; j = 2`
  - `X[2] = 'c'` and `Y[2] = 'c'` → Match ✓. `i = 3; j = 3`
  - `X[3] = 'd'` and `Y[3] = 'd'` → Match ✓. `i = 4; j = 4`
  - `X[4] = 'a'` and `Y[4] = 'f'` → Mismatch ✗. The Brute-Force algorithm does **not** remember the previous successful comparisons. Therefore, move `j` back to the beginning of the pattern, move `i` back to the next possible starting position in the text.
- The first comparison started at **X[0]**, so the next comparison begins at `X[1]`. `Previous start = X[0] → New start = X[1]`
  - `X[1] = 'b'` and `Y[0] = 'a'`. Mismatch ✗. Move to the next starting position (`i = 2; j = 0`)
  - `X[2] = 'c'` and `Y[0] = 'a'` . Mismatch ✗. Move again (`i = 3; j = 0`)
  - `X[3] = 'd'` and `Y[0] = 'a'` . Mismatch ✗. Move again (`i = 4; j = 0`)
- Starting from `X[4]`
  - ```
    X[4] = 'a';  Y[0] = 'a' ✓
    X[5] = 'b' ;  Y[1] = 'b' ✓
    X[6] = 'c';  Y[2] = 'c' ✓
    X[7] = 'a';   Y[3] = 'd' ✗
    ```
  - Mismatch. Restart again from the next position.
- The algorithm continues shifting the pattern one character at a time until it reaches **X[7]**. Starting at X[7]
  - ```
    X[7]  = 'a';   Y[0] = 'a' ✓
    X[8]  = 'b';  Y[1] = 'b' ✓
    X[9]  = 'c';   Y[2] = 'c' ✓
    X[10] = 'd';   Y[3] = 'd' ✓
    X[11] = 'f';   Y[4] = 'f' ✓
    ```
- Every character matches. Therefore, t**he pattern "abcdf" is found in the text, starting at index 7.**

Why is Brute-Force Inefficient?

Notice what happens whenever a mismatch occurs.

```
a b c d a
a b c d f
        ✗
```

Although the first four characters matched successfully, the algorithm completely discards those comparisons. It restarts from the very beginning of the pattern:

- Text: a b c d a b c ...

- Pattern: a b c d f

As a result, many characters are compared repeatedly. This repeated backtracking is what gives the Brute-Force algorithm its worst-case time complexity of $O(n \times m)$, making it inefficient for very large texts.

2.2. Rabin-Karp Algorithm

Unlike the Brute-Force algorithm, the **Rabin-Karp Algorithm** does not immediately compare every character. Instead, it converts both the pattern and every substring of the text into **hash values**.

If two hash values are different, the substrings are guaranteed to be different. Only when the hash values are equal do we perform an actual character-by-character comparison.

This idea significantly reduces unnecessary comparisons. The algorithm makes use of a technique called the **Rolling Hash**, allowing the hash of the next substring to be computed efficiently without recalculating everything from scratch.

The average time complexity is $O(n - m + 1)$, while the worst case (many collisions) becomes $O(nm)$

Example 1 – A Simple Hash Function

- Text `X = "abcdabce"`
- Pattern `Y = "bce"`

For illustration, let the hash function simply be `Hash = sum of ASCII values`

Compute the pattern hash: `Hash("bce") = 98 + 99 + 101 = 298` .

Now compute every substring of length three.

Substring	Hash
abc	294
bcd	297
cda	296
dab	295
abc	294
bce	**298**

Since `Hash("bce") = 298` matches the pattern hash, we compare the actual characters: `b = b ✓; c = c ✓; e = e ✓` → The pattern is found.

Example 2 – Hash Collision

Now consider: Text

- `X = "ccaccaaedba"`
- Pattern `Y = "dba"`

Again, `Hash = ASCII Sum` → Pattern hash: `100 + 98 + 97 = 295`

Substrings

Substring	Hash
cca	295
cac	295
acc	295
caa	293
aae	295
aed	298
edb	299
dba	295

Notice that `cca; cac; acc; aae; dba` all have exactly the same hash value. However, only `dba` is actually the correct pattern. This phenomenon is called a **Hash Collision**.

Improving the Hash Function

A better hash function considers both the character and its position. For a string $P_1P_2P_3...P_m$, the hash can be computed as

$$P_1b^{m-1} + P_2b^{m-2} + ... + P_m$$

Where $b = 26$ for letters, $b = 10$ for digits, or another suitable base.

Using this improved hash, `"dba"= 100 × 10² + 98 × 10¹ + 97 = 11077` . Now the substring hashes become

Substring	Hash
cca	10987
cac	10969
acc	10789
caa	10967
aae	10771
aed	10810
edb	11198
dba	**11077**

Now only `dba` shares the same hash as the pattern.

Hash collisions become much less frequent, making the algorithm significantly more efficient.

Implementation with Python:

```
def rolling_hash_code(x, y, p):  # Return the hash code of X and Y
    res_y = {}
    # Calculate hash code for pattern Y
    n = 0
    for i in range(len(y)):
        n += ord(y[i]) * p ** (len(y) - (i + 1))
    res_y[y] = n  # Append to dictionary
    # Calculate hash code for every sub-string X
    sub_string = []
    for j, k in zip(range(len(x)), range(len(y), len(x) + 1, 1)):
        sub_string.append(x[j:k])
    res_x = {}
    for m in sub_string:
        res_x[m] = sum(ord(i) * p ** (len(y) - (s + 1))
                       for i, s in zip(m, range(len(y))))
    return res_x, res_y


def find_pattern(x, y, p):
    res_x, res_y = rolling_hash_code(x, y, p)
    match = []
    for i in res_x:
        # Find sub-string with the same hash code as the pattern
        if res_x[i] == res_y.get(y):
            match.append(i)
    return match
```

2.3. Knuth-Morris-Pratt (KMP) Algorithm

The **Knuth-Morris-Pratt (KMP) Algorithm** improves upon the Brute-Force algorithm by eliminating unnecessary backtracking.

Instead of restarting the comparison whenever a mismatch occurs, KMP analyzes the **pattern itself** before searching. It computes a table called the **Prefix Function** (also called the **Pi Table** or **LPS Array**) that records how much of the pattern has already been matched.

Using this information, KMP knows exactly where to continue after a mismatch, avoiding repeated comparisons.

The overall time complexity is $O(m + n)$ where $m =$ pattern length; $n =$ text length

- Step 1 – Understanding Prefixes and Suffixes

Consider the pattern `abcdabc` → Prefixes `a; ab; abc; abcd` . Suffixes: `c; bc; abc; dabc`

Notice that `abc` appears as both a prefix and a suffix. KMP uses this repeated structure to avoid restarting from the beginning.

- Step 2 – Construct the Pi Table

Consider pattern = `ababd` . The Pi table is

Index	0	1	2	3	4
Pattern	a	b	a	b	d
Pi	0	0	1	2	0

Interpretation:

- At index 2, the longest proper prefix matching a suffix has length **1**.
- At index 3, it has length **2**.
- At index 4, there is no matching prefix and suffix.

- Step 3 – Search the Text

Suppose `Text = ababcabcabababd; Pattern = ababd` . Initially,

```
Text
ababcabcabababd
^

Pattern
ababd
^
```

Compare

`X[0] = a with Y[0] = a ✓`
`X[1] = b with Y[1] = b ✓`
`X[2] = a with Y[2] = a ✓`
`X[3] = b with Y[3] = b ✓`
`X[4] = c with Y[4] = d ✗`

A mismatch occurs. Instead of returning to the beginning, the Pi table tells us `Pi[3] = 2` . Therefore, the pattern pointer jumps back to position 2 instead of 0. No previously matched characters are compared again.

Continue searching.

`X[2] = a with Y[0] = a ✓`
`X[3] = b with Y[1] = b ✓`
`X[4] = c with Y[2] = a ✗`

Again, the Pi table tells us where to continue immediately. Eventually,

`X[8] = a with Y[0] = a ✓`
`X[9] = b with Y[1] = b ✓`
`X[10] = a with Y[2] = a ✓`
`X[11] = b with Y[3] = b ✓`

`X[12] = 'a' with Y[5] = 'd' ✗, move back to Y[2]`

`X[12] = 'a' with Y[2] = 'a' ✓`

`X[13] = 'b' with Y[3] = 'b' ✓`
`X[14] = d with Y[4] = d ✓`

All characters match. Therefore, t**he pattern is found beginning at index 10.**

Implementation with Python:

```
def kmp_method(S, P):
    m, n = len(P), len(S)  # Initial length for pattern P and string S
    i, j = 0, 0  # Initial pointers i, j pointing the index 0
    Pi = compute_pi(P)  # Find the pi values
    while (n - i) >= (m - j):
        print(f"+ i = {i}, j = {j} => S[i] = {S[i]}, P[j] = {P[j]}")
        if P[j] == S[i]:
            print("S[i] = P[j] -> Match, increment both i and j")
            i += 1
            j += 1
        if j == m:
            print(f"=> Pattern found at {i - j}, and set j as Pi[j - 1]
= {Pi[j - 1]}")
            j = Pi[j - 1]
        # In case of mismatch
        elif i < n and P[j] != S[i]:
            print(f"S[i] != P[j] -> Mismatch -> Set j at Pi[j - 1] = {P
i[j - 1]}. "
                  f"If j returns to index 0 -> increment i")
            if j != 0:
                j = Pi[j - 1]
            else:
                i += 1


def compute_pi(P):  # Creation of Pi Table of Pattern Y
    i, j, m = 1, 0, len(P)  # Length of the match so far
    Pi = [0] * m  # Initial Pi table with Pi values 0
    Pi[0] = 0
    # Calculate Pi[0]
    while i < m:
        # i = {i},  = {j}, P[i], P[j] = {P[i]}, {P[j]}
        if P[i] == P[j]:
            j += 1
            Pi[i] = j
            i += 1
        else:
            if j != 0:
                # P[i] != P[j] = {P[i]} != {P[j]}, and j is not 0,
                # set j as Pi[j - 1], {Pi[j - 1]}
```

```
                j = Pi[j - 1]
            else:
                # P[i] != P[j] = {P[i]} != {P[j]}, and j is 0,
                # set j as Pi[i] = 0 and increment i
                Pi[i] = 0
                i += 1
    return Pi
```

Why is KMP Faster? Consider what happens after a mismatch.

- Brute-Force: Mismatch → Go back to the beginning → Repeat many comparisons
- KMP: Mismatch → Use the Pi table → Continue from the longest valid prefix → No repeated comparisons

The text pointer **never moves backward**, making the algorithm extremely efficient for large texts. This preprocessing step allows KMP to complete the search in $O(m+n)$ time, making it one of the most efficient exact string matching algorithms.

Summary

String Matching is the process of finding a pattern within a larger text and is a fundamental operation in many computer science applications, such as search engines, plagiarism detection, DNA sequencing, and cybersecurity.

Different algorithms solve this problem using different strategies.

- The **Brute-Force algorithm** compares every possible position directly, making it simple but inefficient.
- The **Rabin-Karp algorithm** speeds up searching by comparing hash values before checking characters, though it may suffer from hash collisions.
- The **Knuth-Morris-Pratt (KMP) algorithm** further improves efficiency by preprocessing the pattern and avoiding unnecessary repeated comparisons, achieving an overall time complexity of $O(m+n)$.
- Together, these algorithms demonstrate the evolution from simple searching techniques to more efficient pattern-matching methods.

References

https://drive.google.com/file/d/1QYe1HJ3ypPPLRwudjaFkEFNSwiLVvtW3/view?usp=drive_link

https://www.geeksforgeeks.org/dsa/applications-of-string-matching-algorithms/

https://www.youtube.com/watch?v=qQ8vS2btsxI

https://www.youtube.com/watch?v=V5-7GzOfADQ